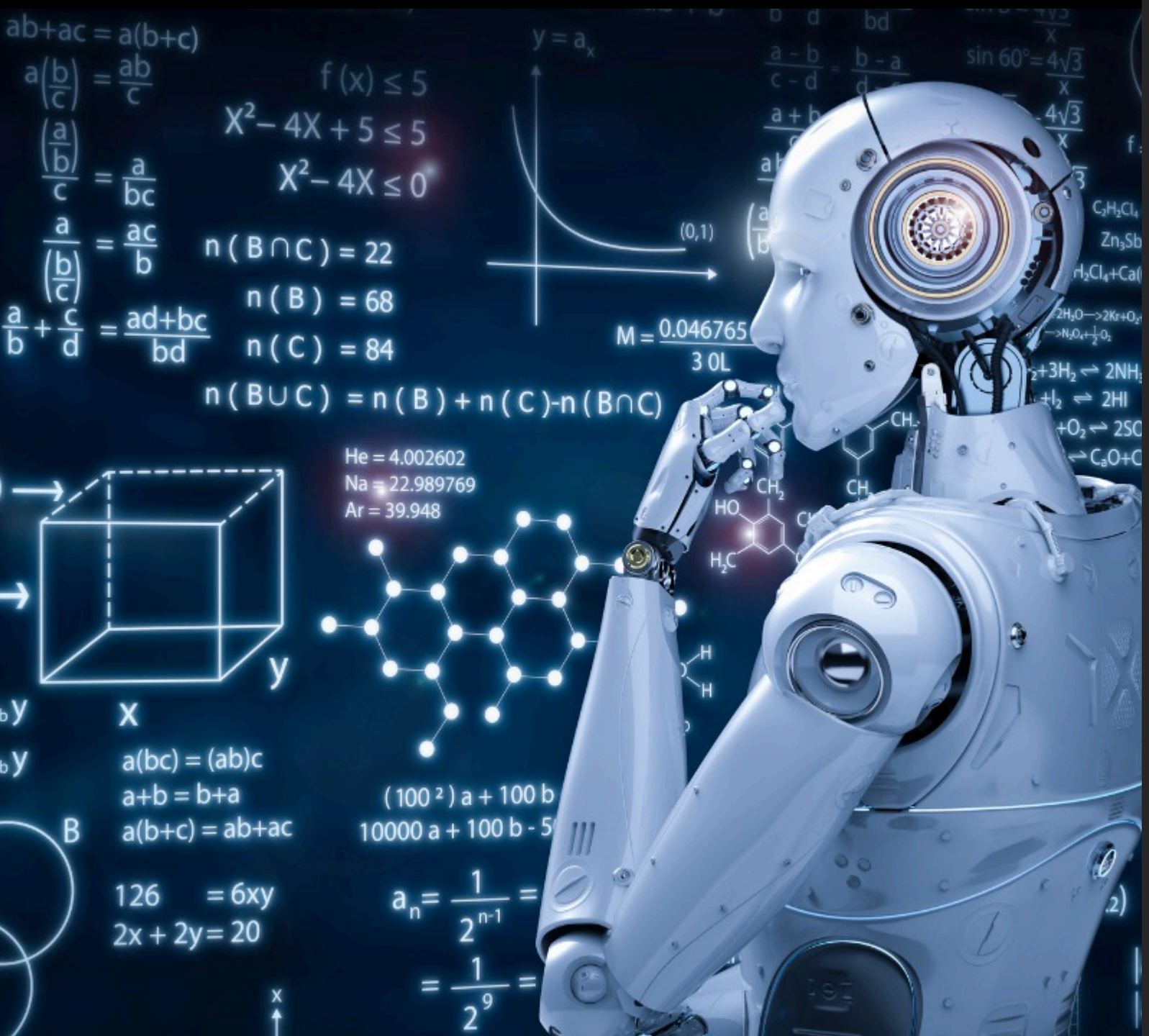


Learning Outcome 3

WEB APPLICATION DEVELOPMENT WITH PYTHON

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

Django Learning Journey

Introduction to Django

- Discover the powerful Python framework that makes web app development faster and easier.

Step-by-Step Setup

- Learn how to install Django and create your very first project, hassle-free.

Build a CRUD Application

- Create a project where you can **add, read, update, and delete data** like a real-world app.

Designing with Templates

- Craft beautiful HTML templates and connect them seamlessly with your Django project.

Mastering Template Tags

- Use Django template tags to dynamically insert data into your HTML pages.

QuerySet Magic

- Learn how to extract, filter, and sort data from your database with ease.

PostgreSQL Integration

- Set up and connect a PostgreSQL database for scalable, production-ready applications.

Deployment Made Simple

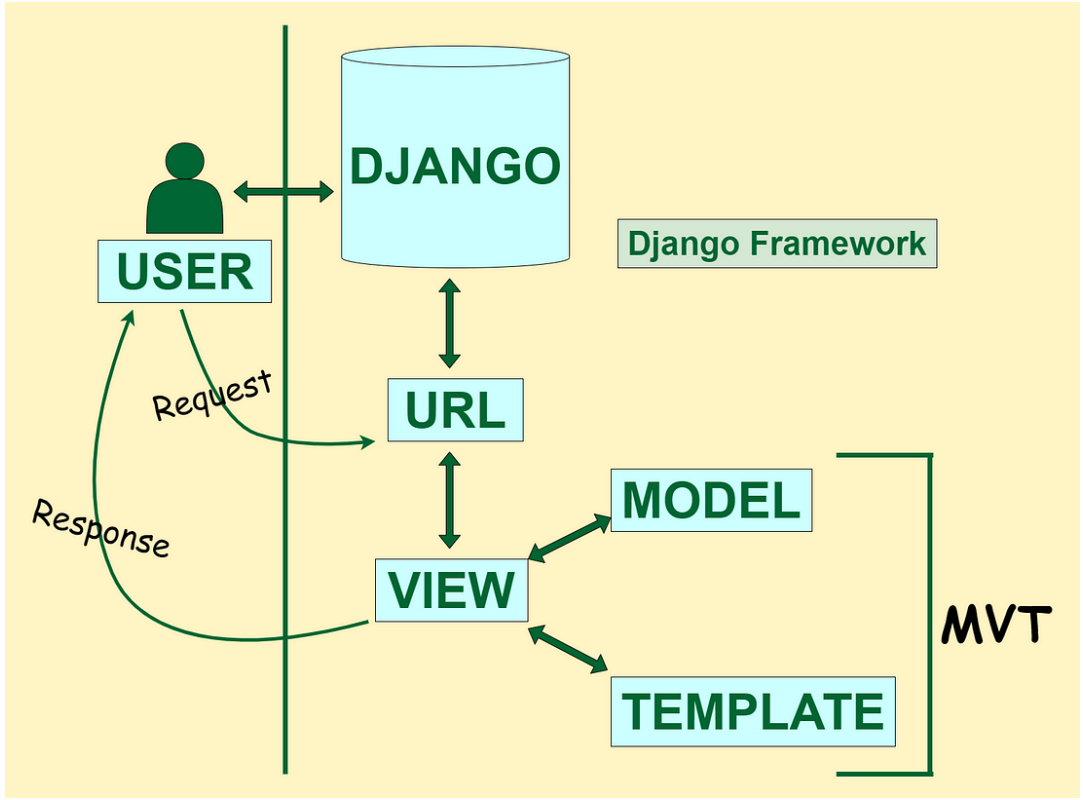
- Deploy your Django project to the world and make it accessible online.

Extra Resource Material

- 🔗 Django Official Site: https://www.djangoproject.com/
- 🔗 Python Official Site: https://www.python.org/
- 🔗 Python Django Tutorial by Dave Gray: Youtube Tutorial Guide

Django

- Django is a free and **open-source, back-end server-side, Python-based web framework** that follows the **model–template–views** (MTV) architectural pattern.
- It is maintained by the **Django Software Foundation (DSF),** an independent organization established in the **US as a non-profit.**
- Django takes care of the difficult stuff so that you can concentrate on building your web applications.
- Django emphasizes the reusability of components, also referred to as **DRY (Don't Repeat Yourself),** and comes with **ready-to-use features like**
  - **login system,**
  - **database connection and**
  - **CRUD operations (Create Read Update Delete).**
- **Django is especially helpful for database driven websites.**
- Some well-known sites that use Django include **Instagram, Mozilla, Disqus, Bitbucket, Nextdoor and Clubhouse.**
- We will follow this tutorial to deeply understand about Django.. HERE. Additionally, you refer Python Django Tutorial by Dave Gray: Youtube Tutorial Gui

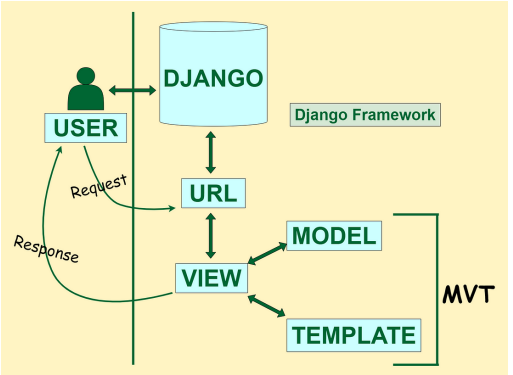


How does Django Work?

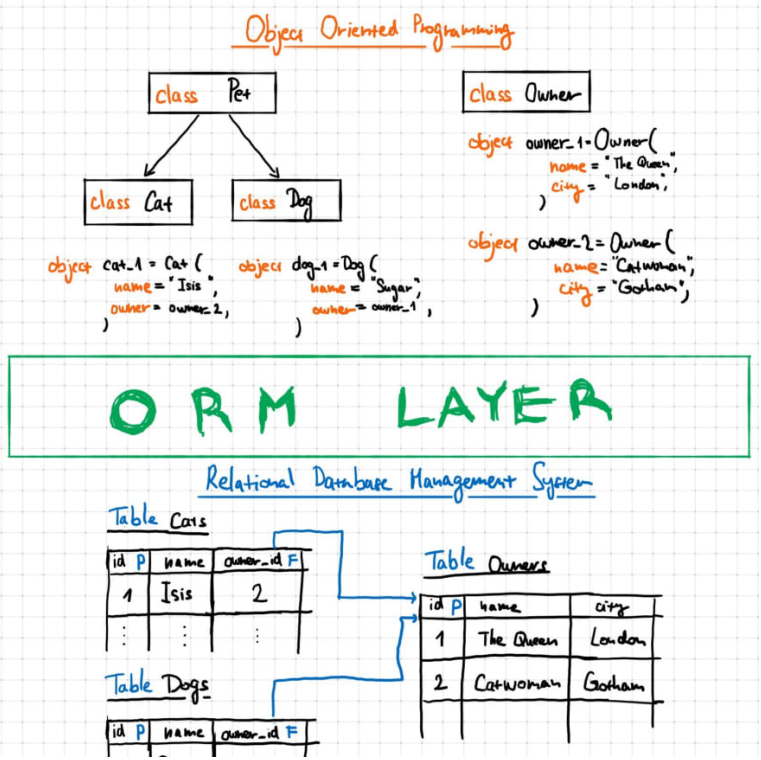
Django follows the **MVT design pattern (Model View Template).**

- **Model -** The data you want to present, usually data from a database.
- **View -** A request handler that returns the relevant template and content - based on the request from the user.
- **Template** - A text file (like an HTML file) containing the layout of the web page, with logic on how to display the data.

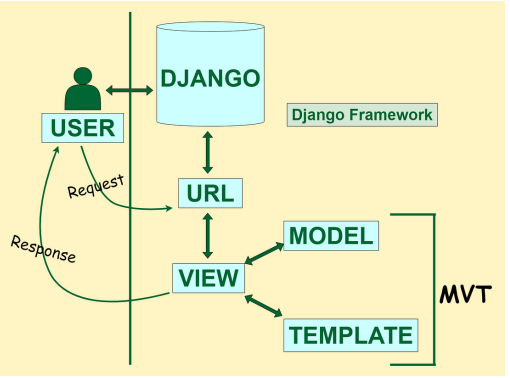
Model



- The model provides data from the database.
- In Django, the data is delivered as an **Object Relational Mapping (ORM)**, which is a technique designed to make it easier to work with databases.
- ORM stands for object-relational mapping, **where objects are used to connect the programming language on to the database systems, with the facility to work SQL and object-oriented programming concepts.**
- The most common way to extract data from a database is SQL. One problem with SQL is that you have to have a pretty good understanding of the database structure to be able to work with it.
- Django, with ORM, makes it easier to communicate with the database, without having to write complex SQL statements.
- The models are usually located in a file called **models.py**
- **Bonus: if you want to dig into understanding this figure in depth:**

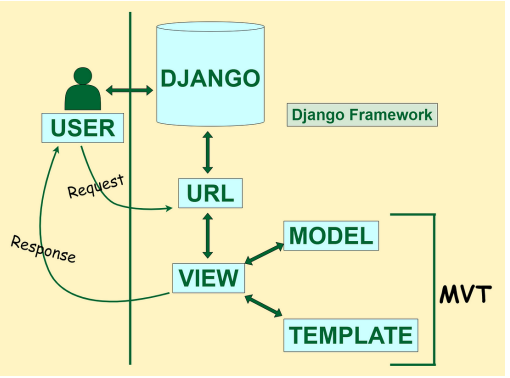


View



- A view is a function or method that
  - takes HTTP requests as arguments,
  - imports the relevant model(s), and
  - finds out what data to send to the template, and returns the final result.
- The views are usually located in a file called views.py

Template

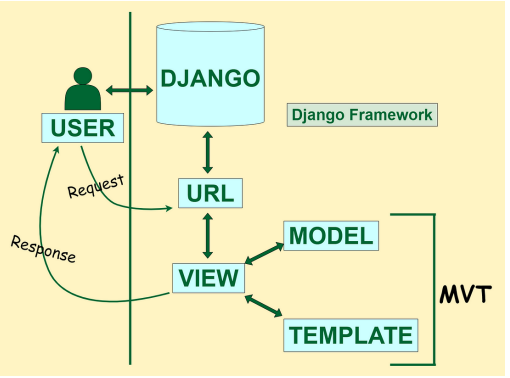


- A template is a file where you describe how the result should be represented.
- Templates are often .html files, with HTML code describing the layout of a web page
- Django uses standard HTML to describe the layout, but uses Django tags to add logic:

```
<h1>My Homepage</h1>

<p>My name is {{ firstname }}.</p>
```

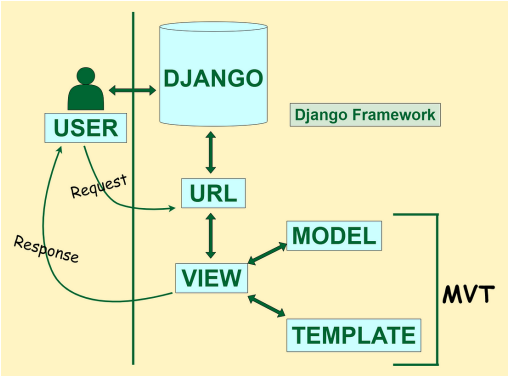
- The templates of an application is located in a folder named **templates**.

URLs



- Django also provides a way to navigate around the different pages in a website.
- When a user requests a URL, Django decides which view it will send it to.
- This is done in a file called **urls.py**

So, What is Going On?



When you have installed Django and created your first Django web application, and the browser requests the URL, this is basically what happens:

- Django receives the URL, checks the urls.py file, and calls the view that matches the URL.
- The view, located in views.py, checks for relevant models.
- The models are imported from the models.py file.
- The view then sends the data to a specified template in the template folder.
- The template contains HTML and Django tags, and with the data it returns finished HTML content back to the browser.

Django Getting Started

- To install Django, you must have Python installed, and a package manager like **PIP**.
- PIP is included in Python from version 3.4.
- **Django Requires Python**
  - To check if your system has Python installed, run this command in the command prompt: **python --version**
  - If Python is installed, you will get a result with the version number, like this: **Python 3.11.4**
  - If you find that you do not have Python installed on your computer, then you can download it for free from the following website: **https://www.python.org/**
- **PIP**
  - To install Django, you must use a package manager like PIP, which is included in Python from version 3.4.
  - To check if your system has PIP installed, run this command in the command prompt: **pip --version**
  - If PIP is installed, you will get a result with the version number, for me on Mac this is like: **pip 23.2.1 from /Users/rashmig/anaconda3/lib/python3.11/site-packages/pip (python 3.11)**
  - If you do not have PIP installed, you can download and install it from this page: **https://pypi.org/project/pip/**
- **Virtual Environment**
  - It is suggested to have a dedicated virtual environment for each Django project, and in the next slides, you will learn how to create a virtual environment, and then install Django in it.

Django - Create Virtual Environment

- It is suggested to have a dedicated virtual environment for each Django project, and one way to manage a virtual environment is **venv**, which is included in Python.
- The name of the virtual environment is your choice, here we will call it **Djtest**.
- Type the following in the command prompt, remember to navigate to where you want to create your project:
  - **Windows:** py -m venv Djtest
  - **Unix/MacOS:** python -m venv Djtest
- This will set up a virtual environment, and **create a folder named "Djtest" with subfolders and files**, like this:

```
(base) rashmig@UIAA107079AB Djtest % ls
bin             include         lib             pyvenv.cfg
```

- Once the environment is activated, you will see this result in the command prompt:
  - **Windows:** (Djtest) C:\Users\Your Name>
  - **Unix/MacOS:** (Djtest) ... $
- **Note:** You must activate the virtual environment every time you open the command prompt to work on your project.

Install Django

- Now, that we have created a virtual environment, we are ready to install Django.
- Note: Remember to install Django while you are in the virtual environment!
- Django is installed using pip, with this command:
  - **Windows:** (Djtest) C:\Users\Your Name>**py** -m pip install Django
  - **Unix/MacOS:** (Djtest) ... $ **python** -m pip install Django
- That's it! Now you have installed Django in your new project, running in a virt
- You can check if Django is installed by asking for its version number like this: (Djtest) C:\Users\Your Name>django-admin –version
- Now you are ready to cr

```
(Djtest) (base) rashmig@UIAA107079AB bin % python -m pip install Django
Collecting Django
  Downloading Django-4.2.6-py3-none-any.whl (8.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.0/8.0 MB 4.0 MB/s eta 0:00:00
Collecting asgiref<4,>=3.6.0 (from Django)
  Downloading asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting sqlparse>=0.3.1 (from Django)
  Downloading sqlparse-0.4.4-py3-none-any.whl (41 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 41.2/41.2 kB 2.7 MB/s eta 0:00:00
Installing collected packages: sqlparse, asgiref, Django
Successfully installed Django-4.2.6 asgiref-3.7.2 sqlparse-0.4.4
```

```
(Djtest) (base) rashmig@UIAA107079AB bin % django-admin --version
4.2.6
```

My First Django Project

- Once you have come up with a suitable name for your Django project, like mine: **my_tennis_club,** navigate to where in the file system you want to store the code (in the virtual environment), and run this command in the command prompt: **django-admin startproject my_tennis_club**

```
(Djtest) (base) rashmig@UIAA107079AB ~ % django-admin startproject my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB ~ % ls
Applications                    OneDrive - Universitetet i Agder
Creative Cloud Files            Pictures
Desktop                         Public
Documents                       PycharmProjects
Downloads                       anaconda3
IdeaProjects                    get-pip.py
Include                         my_tennis_club
Library                         myworl
Movies                          myworld
Music                           test.ipynb
(Djtest) (base) rashmig@UIAA107079AB ~ % cd my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
manage.py       my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % cd my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
__init__.py     asgi.py         settings.py     urls.py         wsgi.py
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

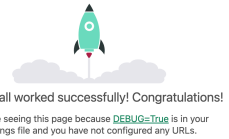
Run the Django Project

- Now that you have a Django project, you can run it, and see what it looks like in a browser.
- Navigate to the /**my_tennis_club** folder and execute this command in the command prompt: **py manage.py runserver**

```
(Djtest) (base) rashmig@UIAA107079AB ~ % cd my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
manage.py       my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
October 06, 2023 - 14:20:34
Django version 4.2.6, using settings 'my_tennis_club.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

- Open a new brow ess ctrl+C

- Congratulations!! We have ke an app in your project. You cannot have a web page created with Django without an app.



Django Create App

- An app is a **web application** that has a specific meaning in your project, like **a home page, a contact form, or a members database.**
- We will now **create an app that allows us to list and register members in a database.**
- But first, let's just create a **simple Django app** that displays **"Hello World!".**

Creating Web App

- Let´s name our app **members**.
- Start by **navigating to the selected location** where you want to store the app, in my case the **my_tennis_club folder**, and run the command below.
- If the server is still running, and you are not able to write commands, press [CTRL] [BREAK], or [CTRL] [C] to stop the server and you should be back in the virtual environment.

```
[06/Oct/2023 14:23:12] "GET / HTTP/1.1" 200 10664
^C%
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
db.sqlite3      manage.py       my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

Creating Web App

- Django creates a folder named members in my project, with this content:
  - **python manage.py startapp members**

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py startapp members
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
db.sqlite3      manage.py       members         my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % cd members
(Djtest) (base) rashmig@UIAA107079AB members % ls
__init__.py     admin.py        apps.py         migrations      models.py       tests.py        views.py
(Djtest) (base) rashmig@UIAA107079AB members % cd migrations
(Djtest) (base) rashmig@UIAA107079AB migrations % ls
__init__.py
(Djtest) (base) rashmig@UIAA107079AB migrations %
```

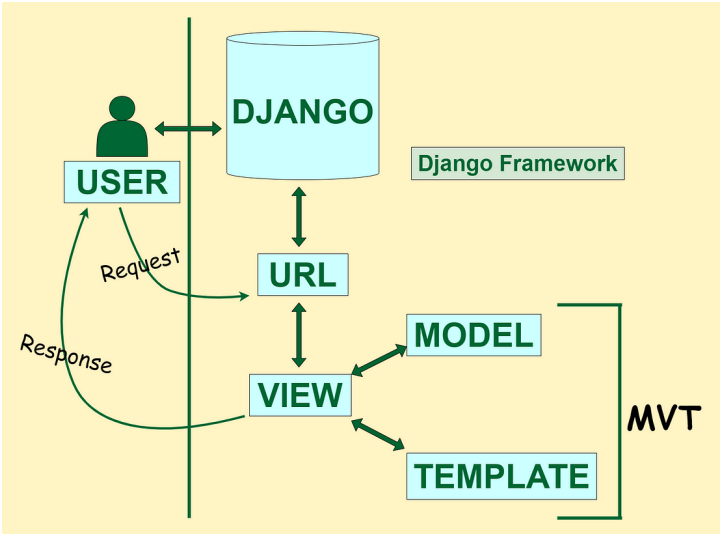
- These are all fil k at the file called **views.py**. This is where we gather the information we need to send back a proper response.

Django Views

- Django views are **Python functions** that **takes http requests** and **returns http response,** like HTML documents.
- A web page that uses Django is full of views with different tasks and missions.
- Views are usually put in a file called **views.py** located on your app's folder.
- There is a views.py in your "members" folder that looks like this:
  - my_tennis_club/members/views.py:
    - python views.py
    - vim views.py
    - type **:w** to enter in write mode
    - type **:q!** to enter in quit without saving any changes
    - Press Esc, type **:x** and hit Enter to save and quit the program.
    - Press Esc, type **:wq** and hit Enter to save the file and exit the editor simultaneously.
- Note: The name of the view does not have to be the same as the application. We call it members because I think it fits well in this context.
- This is a simple example of how to send a response back to the browser.
- But how can we execute the view? Well, we must call the view via a URL.

my_tennis_club/members/views.py:

```
from django.shortcuts import render
from django.http import HttpResponse

def members(request):
    return HttpResponse("Hello world!")
```



Django URLs

- Create a file named **urls.py** in the same folder as the views.py file, and type this code in it:

my_tennis_club/members/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('members/', views.members, name='members'),
]
```

- The urls.py file you [illegible]. We have to do some routing in the root directory **my_tennis_club** as well. This may seem complicated, but for now, just follow the instructions below.
  - There is a file called urls.py on the my_tennis_club folder, open that file and add the include module in the import statement, and also add a path() function in the urlpatterns[] list, with arguments that will route users that comes in via 127.0.0.1:8000/.
  - Then your file will look like this:

my_tennis_club/my_tennis_club/urls.py:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('', include('members.urls')),
    path('admin/', admin.site.urls),
]
```

- If the server is not running, navigate to the /my_tennis_club folder and execute this command in the command prompt: **py manage.py runserver**
- In the browser window, type **127.0.0.1:8000/members/** in the address bar.

Django Templates

- In the Django Intro page, we learned that the result should be in HTML, and it should be created in a template, so let's do that.
- Create a templates folder inside the members folder, and create a HTML file named myfirst.html.
- The file structure should be like this:

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % ls
db.sqlite3      manage.py       members         my_tennis_club
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % cd members
(Djtest) (base) rashmig@UIAA107079AB members % mkdir myfirst.html
(Djtest) (base) rashmig@UIAA107079AB members % ls
__init__.py     __pycache__     admin.py        apps.py         migrations      models.py       myfirst.html    tests.py        urls.py         views.py
(Djtest) (base) rashmig@UIAA107079AB members %
```

- Open the I

my_tennis_club/members/templates/myfirst.html:

```
<!DOCTYPE html>
<html>
<body>

<h1>Hello World!</h1>
<p>Welcome to my first Django project!</p>

</body>
</html>
```

Django Templates: Modify the View

Open the **views.py** file and replace the **members** view with this:

More information about templates.. https://docs.djangoproject.com/en/5.2/intro/tutorial06/

my_tennis_club/members/views.py :

```
from django.http import HttpResponse
from django.template import loader

def members(request):
  template = loader.get_template('myfirst.html')
  return HttpResponse(template.render())
```

Change Settings

- To be able to work with more complicated stuff than "Hello World!", We have to tell Django that a new app is created.
- This is done in the settings.py file in the my_tennis_club folder.
- Look up the INSTALLED_APPS[] list and add the **members** app like this:

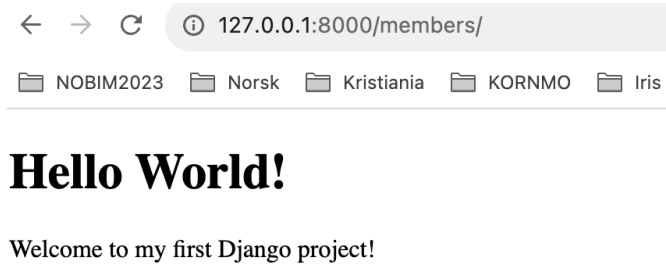
my_tennis_club/my_tennis_club/settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'members'
]
```

- Then run this command: **py mc**
- Which will produce this output:

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

- Start the server by navigating to the /**my_tennis_club** folder and execute this command: py manage
- In the browser window, type **127.0.0.1:8000/members/** in the address bar.



Django Models

- A Django model is the built-in feature that Django uses to create tables, their fields, and various constraints. In short, Django Models is the SQL Database one uses with Django.
- SQL (Structured Query Language) is complex and involves a lot of different queries for creating, deleting, updating, or any other stuff related to a database. Django models simplify the tasks and organize tables into models. Generally, each model maps to a single database table.
- Up until now, output has been static data from Python or HTML templates.
- **Django Models**
  - Create a model in Django to store data in the database conveniently. Moreover, we can use the admin panel of Django to create, update, delete, or retrieve fields of a model and various similar operations. Django models provide simplicity, consistency, version control, and advanced metadata handling. The basics of a model include –
  - Each model is a Python class that subclasses **django.db.models.Model.**
  - Each attribute of the model represents a database field.
  - With all of this, Django gives you an automatically generated database-access API. Now we will see how Django allows us to work with data, without having to change or upload files in the process.

Create Table (Model)

- To create a model, navigate to the **models.py** file in the /**members**/ folder.
- Open it, and add a **Member** table by creating a **Member class**, and describe the table fields in it:

my_tennis_club/members/models.py:

```
from django.db import models

class Member(models.Model):
  firstname = models.CharField(max_length=255)
  lastname = models.CharField(max_length=255)
```

- The first field, firstnam first name of the members.
- The second field, lastname, is also a Text field, with the member's last name.
- Both firstname and lastname is set up to have a maximum of 255 characters.
- **SQLite Database**
  - When we created the Django project, we got an **empty SQLite database**.
  - It was created in the **my_tennis_club root folder, and has the filename db.sqlite3**.
  - By default, all Models created in the Django project will be created as tables in this database.

Migrate

- Now when we have described a Model in the models.py file, we must run a command to actually create the table in the database.
- Navigate to the /**my_tennis_club/ folder** and run this command: py manage.py makemigrations members

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py makemigrations members
Migrations for 'members':
  members/migrations/0001_initial.py
    - Create model Member
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

- Django creates a file describing tl

```
# Generated by Django 4.2.6 on 2023-10-06 17:49

from django.db import migrations, models


class Migration(migrations.Migration):
    initial = True

    dependencies = []

    operations = [
        migrations.CreateModel(
            name="Member",
            fields=[
                (
                    "id",
                    models.BigAutoField(
                        auto_created=True,
                        primary_key=True,
                        serialize=False,
                        verbose_name="ID",
                    ),
                ),
                ("firstname", models.CharField(max_length=255)),
                ("lastname", models.CharField(max_length=255)),
            ],
        ),
    ]
~
~
```

Migrate

- Note that Django inserts an id field for your tables, which is an auto increment number (first record gets the value 1, the second record 2 etc.), this is the default behavior of Django, you can override it by describing your own id field.
- The table is not created yet, you will have to run one more command, then Django will create and execute an SQL statement, based on the content of the new file in the /migrations/ folder.
- Run the migrate command: **py manage.py migrate**
- Which will result in this output:

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, members, sessions
Running migrations:
  Applying members.0001_initial... OK
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

- Now you have a **Member**

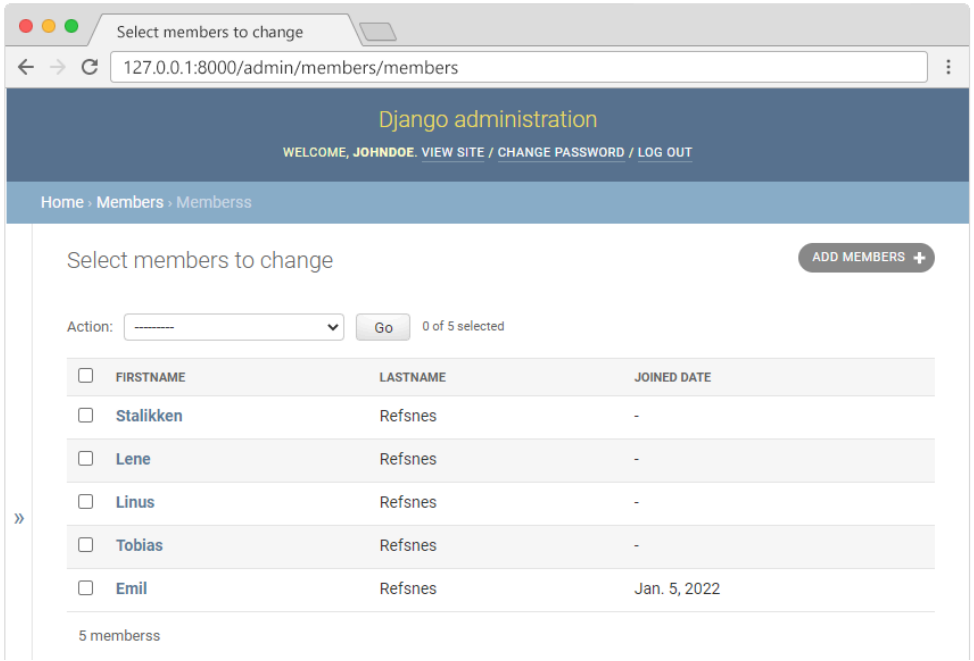
View SQL

- As a side-note: you can view the SQL statement that were executed from the migration before.
- All you have to do is to run this command, with the migration number: **py manage.py sqlmigrate members 0001**
- Which will result in this output:

```
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club % python manage.py sqlmigrate members 0001
BEGIN;
--
-- Create model Member
--
CREATE TABLE "members_member" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "firstname" varchar(255) NOT NULL, "lastname" varchar(255) NOT NULL);
COMMIT;
(Djtest) (base) rashmig@UIAA107079AB my_tennis_club %
```

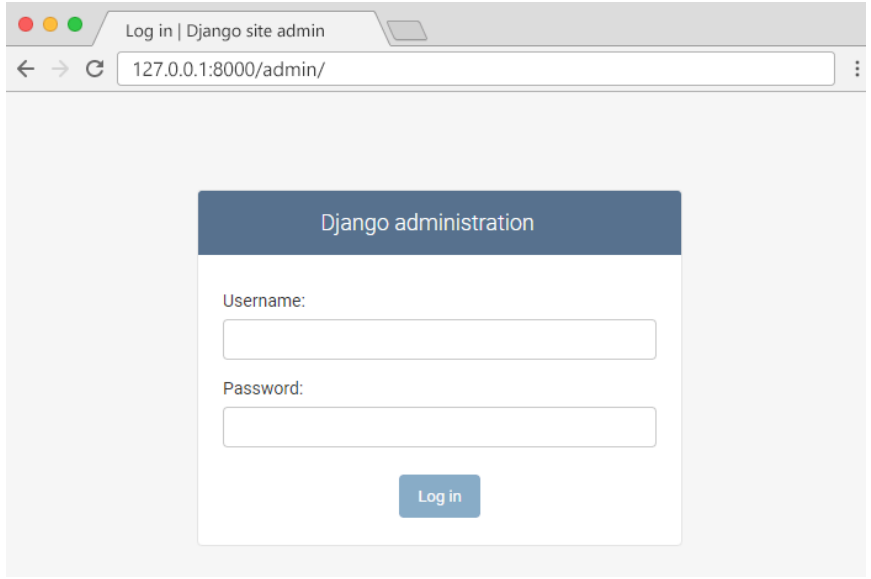
Django Admin

- Django Admin is a really great tool in Django, it is actually a CRUD* user interface of all your models!
- *CRUD stands for Create Read Update Delete.
- It is free and comes ready-to-use with Django:



Getting Started

- To enter the admin user interface, start the server by navigating to the / **yourproject folder** and execute this command: **py manage.py runserver**
- In the browser window, type **127.0.0.1:8000/admin/** in the address bar. The result should look like this:



Getting Started

- The reason why this URL goes to the Django admin log in page can be found in the urls.py file of your project:

my_tennis_club/my_tennis_club/urls.py:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('', include('members.urls')),
    path('admin/', admin.site.urls),
]
```

- The urlpatterns[] list takes reques[illegible]te.urls, which is part of a built-in application that comes with Django, and contains a lot of functionality and user interfaces, one of them being the log-in user interface.

Django Admin - Create User

- To be able to log into the admin application, we need to create a user.
- This is done by typing this command in the command view:

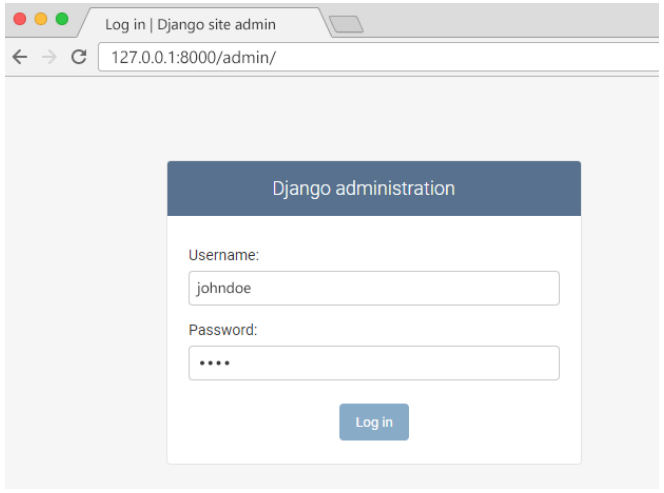
py manage.py createsuperuser

- Which will give this prompt and you must enter: username, e-mail address, (you can just pick a fake e-mail address), and password:
-

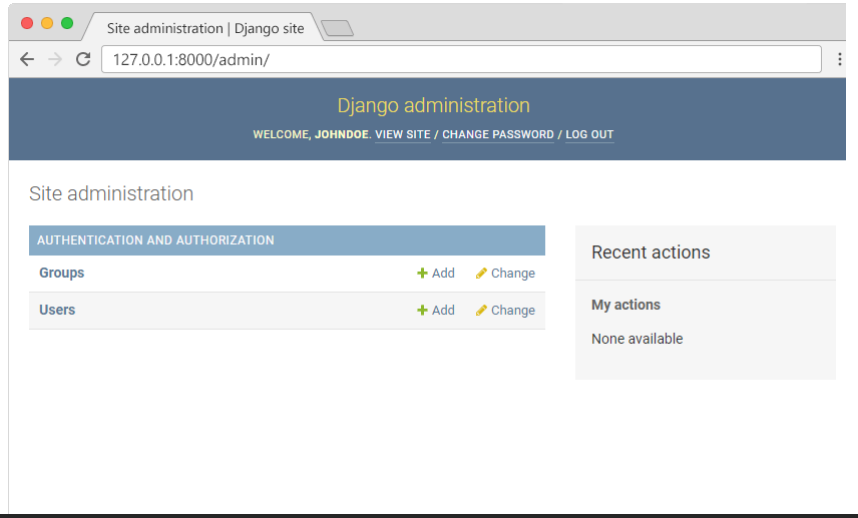
```
Username: johndoe
Email address: johndoe@dummymail.com
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]:
```

Django Admin - Create User

- Now start the server again: **py manage.py runserver**
- And fill in the form with the correct username and password:
- Here you can create, read, update, and delete groups and users, but **where is the Members model?**



Which should result in this user interface



Django Admin - Include Member in the Admin Interface

To include the Member model in the admin interface, we have to tell Django that this model should be visible in the admin interface.

This is done in a file called **admin.py**, and is located in your app's folder, which in our case is the **members folder**.

Open it, and it should look like this:

my_tennis_club/members/admin.py:

```
from django.contrib import admin

# Register your models here.
```

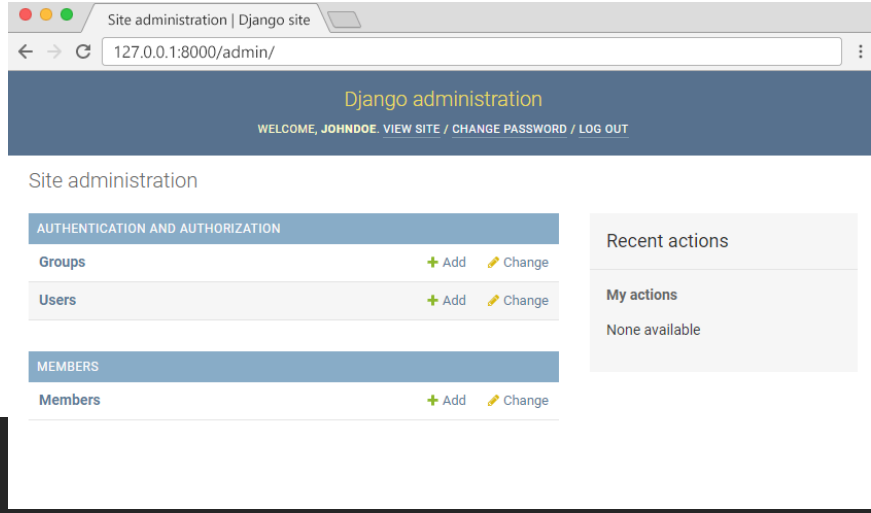
Django Admin - Include Member in the Admin Interface

- Insert a couple of lines here to make the Member model visible in the admin page:

my_tennis_club/members/admin.py:

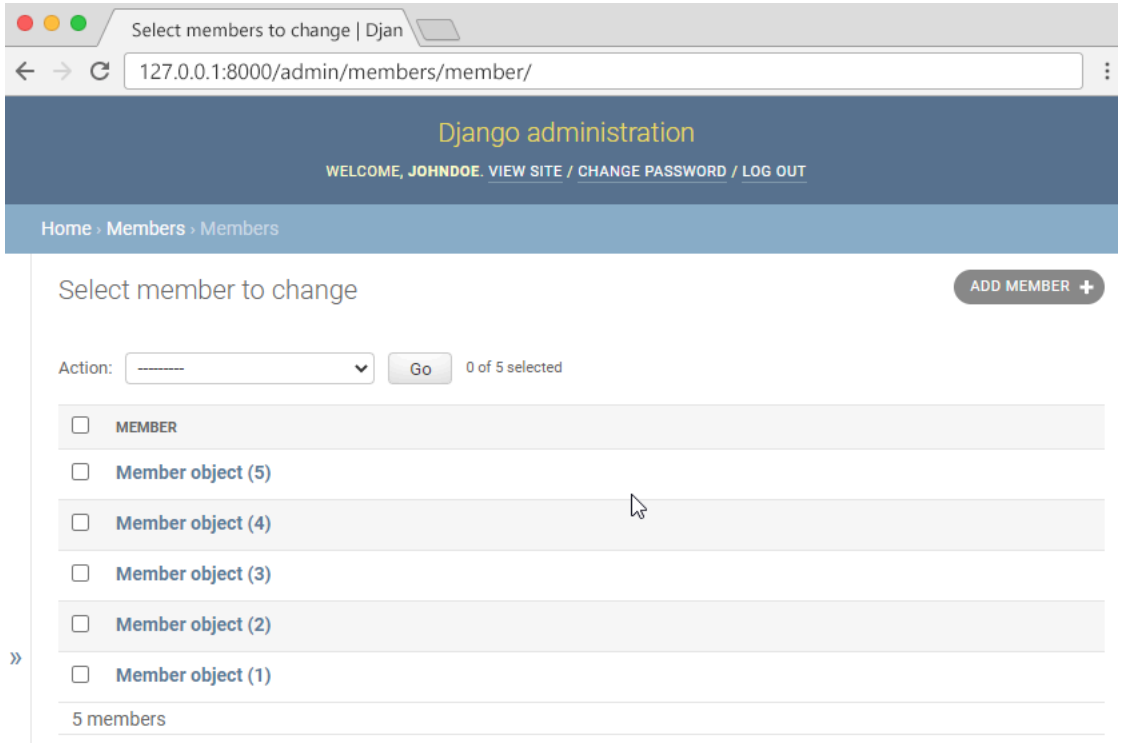
```
from django.contrib import admin
from .models import Member

# Register your models here.
admin.site.register(Member)
```

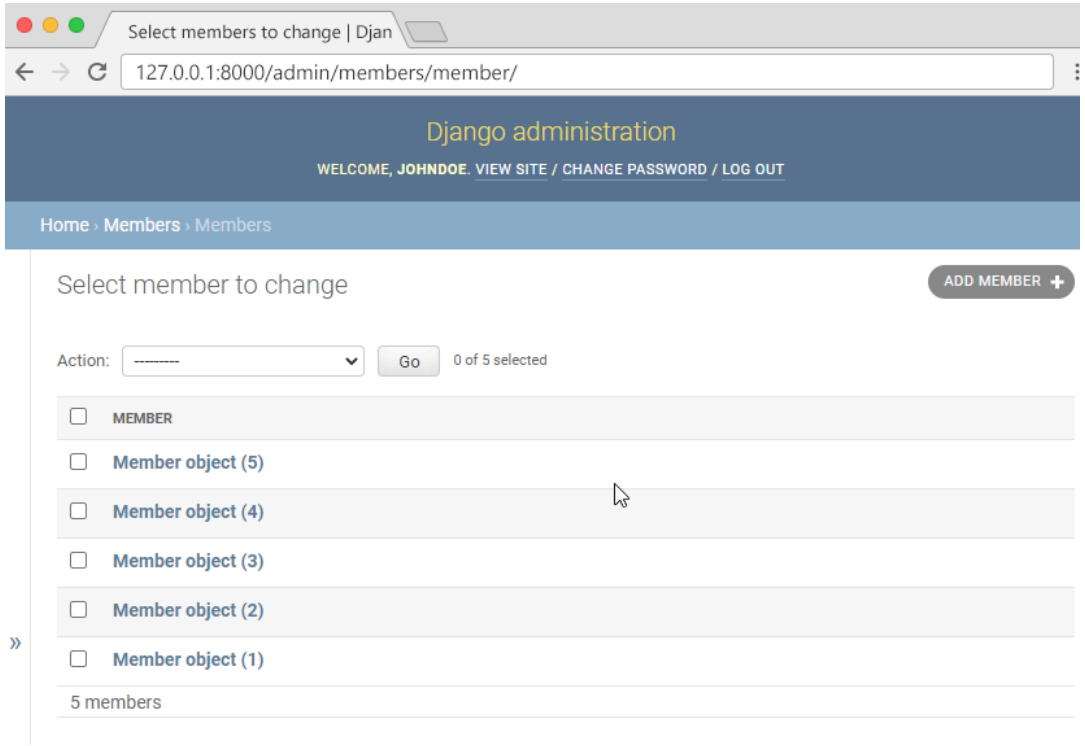
- Now go back to the browser and you should get this result:



Django Admin - Include Member in the Admin Interface

Click **Members** and see the **five records we inserted earlier** in this lecture:





Django Admin - Set Fields to Display

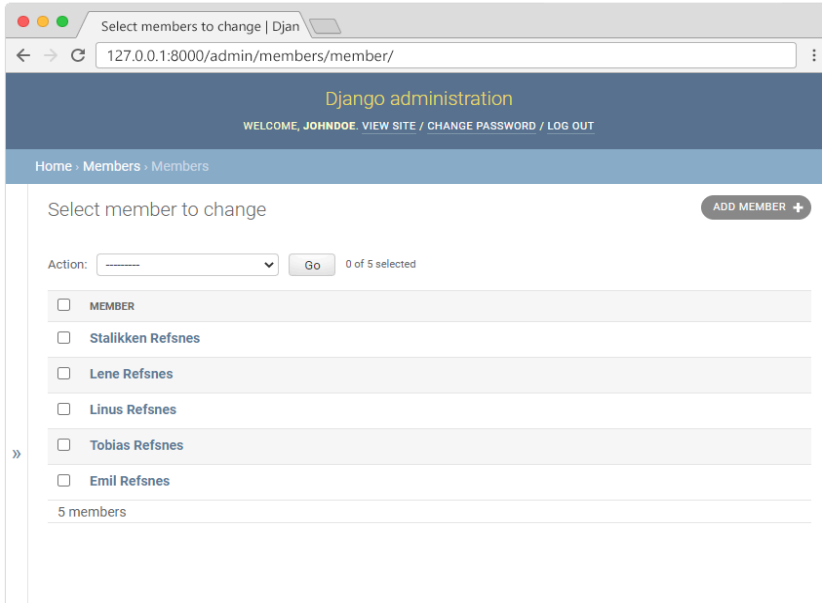
- **Make the List Display More Reader-Friendly**
  - When you display a Model as a list, Django displays each record as the string representation of the record object, which in our case is "Member object (1)", "Member object(2)" etc.:
  - To change this to a more reader-friendly format, we have two choices:
    - Change the string representation function, **__str__()** of the Member Model
    - Set the **list_details** property of the Member Model

Change the String Representation Function

we have to define the **__str__() function** of the **Member Model in models.py**, like this:

my_tennis_club/members/models.py:

```
from django.db import models

class Member(models.Model):
  firstname = models.CharField(max_length=255)
  lastname = models.CharField(max_length=255)
  phone = models.IntegerField(null=True)
  joined_date = models.DateField(null=True)

  def __str__(self):
    return f"{self.firstname} {self.lastname}"
```

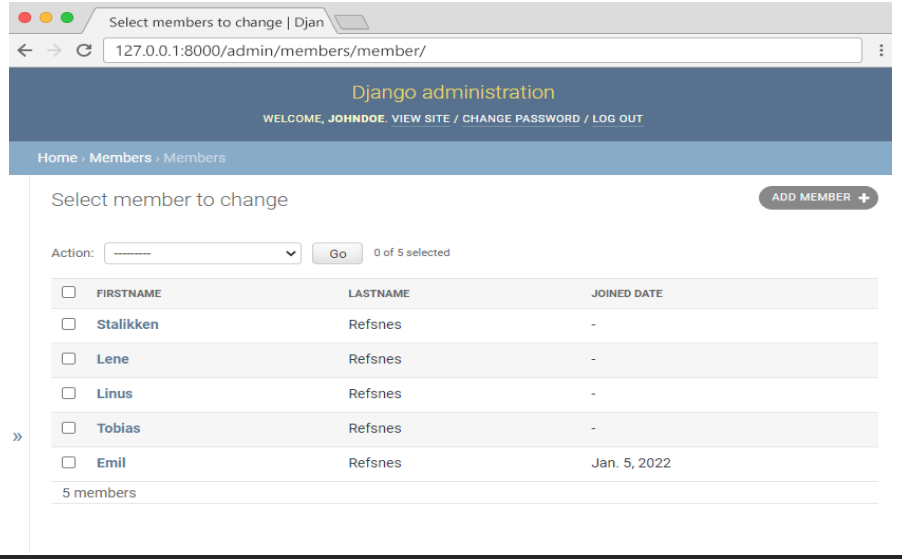


Set list_display

- We can control the fields to display by specifying them in a **list_display property in the admin.py file.**
- First create a **MemberAdmin() class** and specify the **list_display tuple,** like this:
- Remember to add the **MemberAdmin as an argumet** in the **admin.site.register(Member, MemberAdmin).**
- Now go back to the browser and you should get this result:

my_tennis_club/members/admin.py:

```
from django.contrib import admin
from .models import Member

# Register your models here.

class MemberAdmin(admin.ModelAdmin):
  list_display = ("firstname", "lastname", "joined_date",)

admin.site.register(Member, MemberAdmin)
```



Django Admin – Update, Add, and Delete Members

- Now we are able to create, update, and delete members in our database, and we start by giving them all a date for when they became members.
- Click the first member, Stalikken, to open the record for editing, and give him a joined_date: